

STARBLAST — Development Reflection

CS Game Development Project | 2025

Project Summary

STARBLAST is a top-down arcade space shooter developed entirely in C using the raylib graphics library, compiled under MSYS2 MinGW64 UCRT. The project challenged our team to apply systems-level programming, game architecture design, and real-time collision logic — all within a single-semester timeline.

Architecture & Modularity

We divided the codebase into seven purpose-focused modules, each with its own header and implementation file:

Module	Responsibility
player.c/.h	Input handling, ship movement, shooting cooldown, mercy invincibility
enemy.c/.h	Wave spawning, AI movement (bounce + boss tracking), enemy shooting
bullet.c/.h	Bullet pool management, position update, out-of-bounds culling
particle.c/.h	Particle system with per-particle lifetime, velocity drag, alpha fade
collision.c/.h	AABB/radius collision between bullets↔enemies and bullets↔player
background.c/.h	Scrolling parallax starfield using deterministic LCG PRNG
ui.c/.h	HUD rendering, all menu screens (main/pause/gameover/win)

Centralising shared types in game.h (the Game context struct, enums, and constants) meant every module could be included independently without circular dependencies. This separation of concerns made parallel development straightforward — one team member could work on enemy AI while another refined the particle system.

Key Technical Challenges

Bullet pool allocation:

Heap allocation per bullet caused measurable frame hitches at high fire rates. We switched to a fixed-size pool (MAX_BULLETS = 64) with an active flag. Scanning for the first inactive slot is O(n) but n is small and cache-friendly.

Deterministic PRNG:

raylib does not expose seeded randomness. We implemented a simple LCG (seed x 1664525 + 1013904223) to keep particle and enemy spawn patterns reproducible across platforms without importing the stdlib rand().

Boss difficulty tuning:

Early playtests showed the boss was either trivially easy or instant death. We iterated on spread angle, fire rate scaling by wave, and the HP multiplier until the boss felt challenging but fair across all 5 waves.

Mercy invincibility:

Without post-hit invincibility, a single enemy bullet volley could chain-kill the player in one frame. We added a 1.5-second invincible window with a visible flicker effect (toggling draw every 0.1 s) to communicate the state to the player.

What Went Well

The modular design proved its value immediately. Because each subsystem owned its own data (e.g., the particle array lives in `Game` but is only written by `particle.c`), debugging was largely isolated — a crash in `enemy_update_all` could not corrupt player state by accident. The Makefile's automatic object-file discovery via wildcard meant adding a new `.c` file required zero Makefile edits, which reduced friction during the rapid prototyping phase.

raylib's immediate-mode drawing API was a strong fit for a C project: no scene graph, no retained objects, just draw calls each frame. This kept our architecture simple and deterministic. The lack of a built-in audio system was not a problem for our scope — we kept the game purely visual for this submission.

What We Would Improve

Sound effects: All explosion and laser feedback is currently visual only. raylib's `InitAudioDevice` / `LoadSound` API would let us add retro SFX without external dependencies — a natural next step.

High-score persistence: Scores reset on exit. Writing the top-5 scores to a local `scores.txt` file with `fopen/fwrite` would add meaningful replay motivation.

Dynamic difficulty: Wave scaling is currently linear (count ± 3 per wave). A per-enemy individual behaviour budget — increasing speed, accuracy, and aggression independently — would feel less mechanical.

Sprite-based rendering: Ship and enemy shapes are drawn with primitive shapes (triangles, rectangles). Replacing these with raylib `Texture2D` sprites loaded from PNG files would greatly improve visual polish without restructuring the architecture.

Two-player co-op: The architecture could support a second `Player` struct; `collision_check` would simply iterate over both. Adding a second player controlled by IJKL keys is an achievable extension that would have added significant replay value.

Team Reflection

Working in C rather than a higher-level language forced us to think carefully about memory layout and ownership from the start. The `game.h` central context struct acted as a contract between modules — any team member could read it to understand the full game state at a glance. This discipline paid off: we had zero memory leaks (verified by inspecting all `allocs` — there are none, all data is stack/static) and no undefined-behaviour crashes in final testing.

The biggest lesson was the importance of playtesting early. The game's 'feel' — movement speed, bullet speed, enemy aggression — took far more iteration than the code itself. Numbers that seemed correct in isolation (e.g., 400 px/s bullet speed) felt wrong in play. We scheduled a dedicated tuning session in week 13 that transformed the game from technically functional to genuinely enjoyable.

Conclusion

STARBLAST met all project requirements: C-only codebase, proper modularity, MSYS2-compatible Makefile, and a complete game loop with menus, multiple enemy types, lives, scoring, and a win condition. Beyond the spec, the project gave us hands-on experience with real-time game architecture patterns — fixed-size object pools, state machines, particle systems — that translate

directly to professional game and systems programming contexts.